

Deploying AI Assistants to Production

Shipping the Neo Nexus assistant on Streamlit in Snowflake: structure, environments, CI/CD with an evaluation gate, least-privilege security and day-two operations

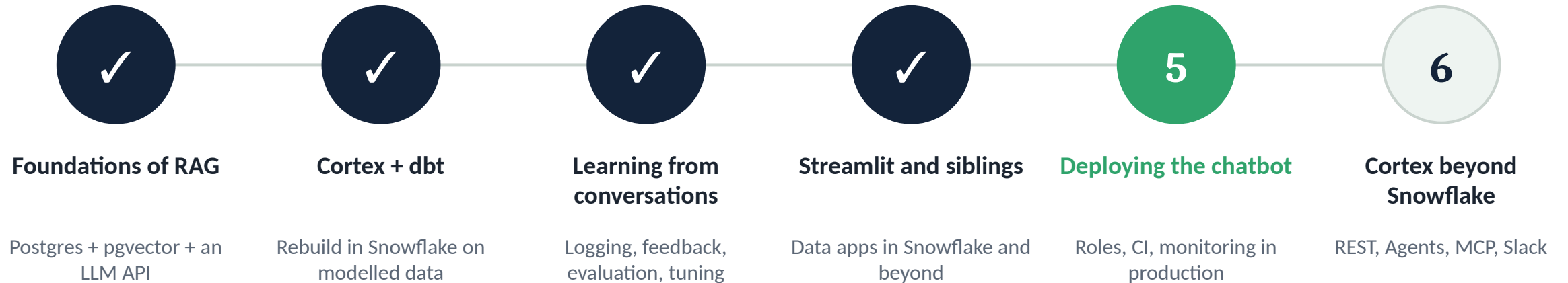
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



From a working app to a production service



Today's output: a Git repository, a CI pipeline with an evaluation gate, production roles, a health view with alerts, and a rehearsed rollback.

By the end of today you can...



Structure

the assistant as a multi-page app with thin UI, testable logic and environment-driven configuration.



Automate

deployment with the Snowflake CLI or Git integration, and block regressions with an evaluation gate in CI.



Secure

the service with least-privilege roles for the app, its users, its admins and the pipeline.



Operate

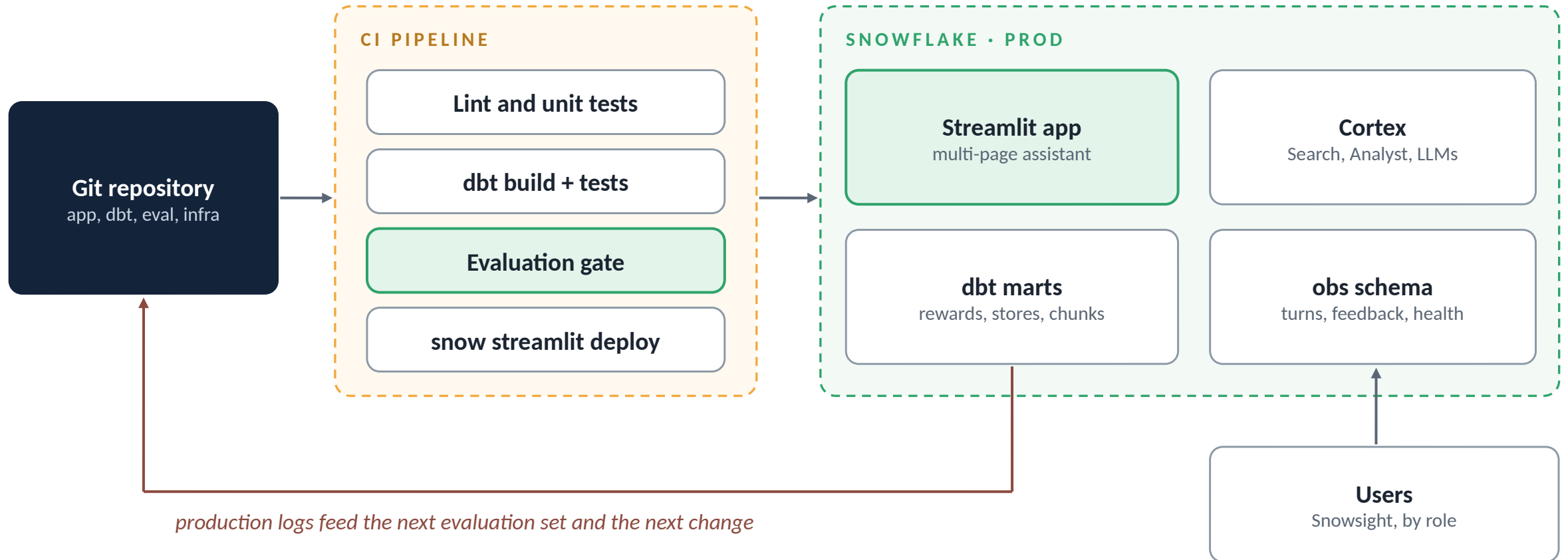
it with a health view, alerts, cost controls, a runbook and a tested rollback.

What changes between a demo and a service

Concern	Demo (Module 4)	Production (today)
Code	Files uploaded from a laptop	Git repository, reviewed pull requests
Deployment	CREATE STREAMLIT by hand	CI pipeline, identical every time
Environments	One schema	DEV, TEST and PROD from one codebase
Quality	It looked right in the demo	Evaluation gate blocks regressions
Access	Owner's role, broad grants	Least-privilege roles per purpose
Operations	Nobody is watching	Health view, alerts, runbook, rollback
Cost	Unbounded	Resource monitors and budgets

None of these rows is about the model. The production gap is engineering and operations.

Git to pipeline to Snowflake, with a loop back from production



Every box here is covered today.

01

Structure the app

A repository and app layout built for change: thin UI, testable logic, configuration by environment, graceful failure

One repository holds everything the assistant needs

REPOSITORY

```
neo-nexus-assistant/  
├── app/  
│   ├── streamlit_app.py      # entry point  
│   └── pages/  
│       ├── 1_Assistant.py  
│       ├── 2_Rewards_explorer.py  
│       └── 3_Admin_feedback.py  
│   └── lib/                  # no Streamlit here  
│       ├── answer.py  
│       ├── session.py  
│       └── logging.py  
├── environment.yml           # pinned packages  
├── snowflake.yml  
├── dbt/                      # Module 2  
├── eval/                    # Module 3 gate  
├── tests/                   # unit tests for lib/  
└── .github/workflows/deploy.yml
```

Thin UI, thick lib

pages only render; lib/ holds logic you can unit-test

One change, one PR

app, dbt model and prompt ship together, reviewed together

Pinned packages

environment.yml: production runs what was tested

Pages per audience

analysts chat and explore; admins review feedback

One codebase, three environments, zero hard-coded names

SNOWFLAKE CLI PROJECT

```
# app/snowflake.yml
definition_version: 2
env:
  DB: NEO_NEXUS_DEV           # overridden in CI
  WH: APP_DEV_WH
entities:
  assistant:
    type: streamlit
    identifier:
      name: neo_nexus_assistant
      database: <% ctx.env.DB %>
      schema: APPS
    query_warehouse: <% ctx.env.WH %>
    main_file: streamlit_app.py
    pages_dir: pages/
    stage: app_stage
    artifacts:
      - streamlit_app.py
      - pages/
      - lib/
      - environment.yml
```

DEV

NEO_NEXUS_DEV
developers, sample data

TEST

NEO_NEXUS_TEST
CI deploys; gate runs here

PROD

NEO_NEXUS_PROD
users; deploy needs approval

LIB/SESSION.PY

```
# inside the app: never hard-code
DB = session.get_current_database()
```

Stream answers, and fail politely

PAGES/1_ASSISTANT.PY

```
with st.chat_message("assistant"):
    try:
        stream = complete(ANSWER_MODEL, prompt,
                           session=session, stream=True)
        text = st.write_stream(stream)      # words appear live
        log_turn(q, text, status="ok")
    except Exception as e:                  # timeout, throttle,
        text = ("Sorry, I can't answer that right now. "
               "Please try again, or contact the "
               "loyalty support team.")
        st.warning(text)
        log_turn(q, None, status="error",    # counted in
                  error=type(e).__name__)   # obs.daily_health
```

Stream

First words in about a second; the full answer still takes as long, but feels fast.

Fail politely

A clear message and a human fallback; the error is logged, never shown as a stack trace.

02

Automate

Two ways to deploy, one pipeline with an evaluation gate, and a rollback you have rehearsed

Push from CI, or let Snowflake pull from Git

A • Snowflake CLI from CI

CI uploads the files and runs snow streamlit deploy. Works with any Git host and CI tool. Used in today's lab.

B • Snowflake Git repository

Snowflake holds a clone of the repository; the app is created from a branch or tag, refreshed with FETCH. Private repositories need GIT_CREDENTIALS pointing at a secret.

SNOWFLAKE SQL

```
-- path B: Snowflake pulls from Git
create api integration git_api
  api_provider = git_https_api
  api_allowed_prefixes =
    ('https://github.com/neo-nexus/')
  enabled = true;

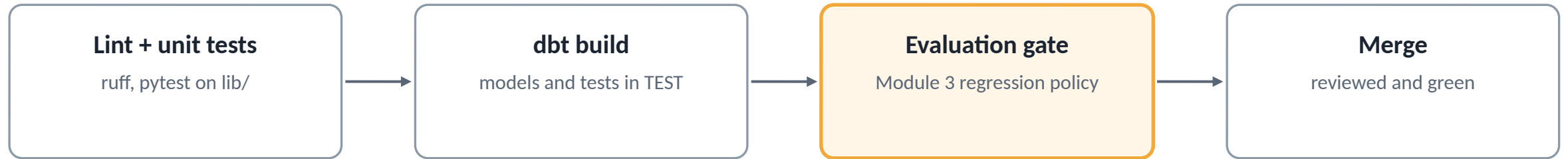
create git repository apps.assistant_repo
  api_integration = git_api
  origin =
    'https://github.com/neo-nexus/assistant.git';

alter git repository apps.assistant_repo fetch;

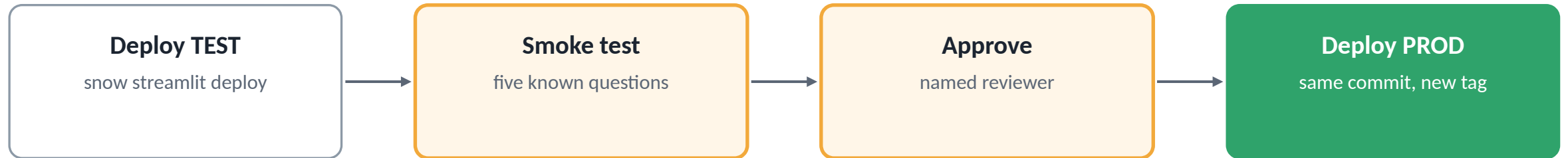
create or replace streamlit apps.neo_nexus_assistant
  from @apps.assistant_repo/branches/main/app
  main_file      = 'streamlit_app.py'
  query_warehouse = app_prod_wh;
```

Every change passes the same gates, in the same order

ON PULL REQUEST



ON MERGE TO MAIN



Build once, promote many: the commit that passed the gates is exactly the one users get.

The pipeline as code: GitHub Actions and the Snowflake CLI

`.GITHUB/WORKFLOWS/DEPLOY.YML`

```
name: deploy-assistant
on:
  push:
    branches: [main]
permissions:
  id-token: write          # OIDC, no stored keys
  contents: read
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: prod      # requires an approver
    steps:
      - uses: actions/checkout@v4
      - uses: snowflakedb/snowflake-actions@v3
        with:
          use-oidc: true
      - name: Evaluation gate
        run: python eval/run_gate.py --target PROD
      - name: Deploy app
        working-directory: app
        env: { DB: NEO_NEXUS_PROD, WH: APP_PROD_WH }
        run: snow streamlit deploy --replace
```

Official action

snowflakedb/snowflake-actions installs the CLI

OIDC, not secrets

no long-lived Snowflake credentials in GitHub

Approval gate

the prod environment needs a named reviewer

Same file, all envs

only DB and WH change

Module 3's regression policy, as a script that can say no

PYTHON

```
# eval/run_gate.py
results = run_eval_set(session, target=args.target) # Module 3
scores = judge(session, results) # per slice
base = session.table("eval.baseline").to_pandas()

drop = (base.set_index("slice")[METRICS]
        - scores.set_index("slice")[METRICS])

failed = []
if (drop > 0.02).any().any(): # scores 0 to 1
    failed.append(f"regression:\n{drop.round(3)}")
if scores["adversarial_pass"].min() < 1.0:
    failed.append("adversarial case failed")
if scores["cost_per_answer"].mean() > 1.10 * base_cost:
    failed.append("cost up more than 10%")

if failed:
    print("\n".join(failed)); sys.exit(1) # block deploy
save_candidate_scores(session, scores)
```

Quality

no slice drops more than 0.02

Safety

every adversarial case passes

Cost

cost per answer rises no more than 10%

Know how to undo every kind of change before you ship it

What changed	How to roll back	Speed
App code	redeploy the previous Git tag through the pipeline	minutes
Prompt version	set the previous version active in app.prompts	seconds
Model choice	change the model string in the config table	seconds
dbt models / data	revert the PR and rebuild; Time Travel for data	minutes to hours
Search service	keep the old service; switch the app back to it	seconds, if prepared

Design rule: prompts and model choices are configuration, not code, so the fastest rollbacks need no deployment.

03

Secure

Least-privilege roles for the app, its users, its admins and the pipeline, and protecting the data in conversations

Four roles, each with only what its job needs

APP_OWNER

owns the app; reads marts, uses search and semantic view, writes obs

CRM_ANALYST

USAGE on the app; nothing else

ASSISTANT_ADMIN

reads obs for the admin feedback page

CI_DEPLOYER

creates and replaces apps in APPS; no data access

SNOWFLAKE SQL

```
-- the app's owner: exactly what it needs
grant usage on cortex search service
    serving.loyalty_search          to role app_owner;
grant select on semantic view
    serving.rewards_sv              to role app_owner;
grant select on all tables in schema marts
    to role app_owner;
grant insert on all tables in schema obs
    to role app_owner;

-- users: open the app, nothing more
grant usage on streamlit apps.neo_nexus_assistant
    to role crm_analyst;

-- the pipeline: deploy, never read data
grant create streamlit on schema apps
    to role ci_deployer;
```

Conversations are customer data: treat them that way



Mask free text

masking policy on question and answer columns; only admins see raw text



Retain, then delete

a scheduled task removes turns older than the agreed retention period



No outbound access

no external access integration: data cannot leave through the app



Audit

query and access history show who read which conversations

04

Operate

Health views, alerts, cost controls and a runbook: what keeps the assistant good after launch day

One view answers: is the assistant healthy today?

SNOWFLAKE SQL

```
create or replace view obs.daily_health as
select
  date_trunc('day', t.asks_at)           as day,
  count(*)                             as turns,
  count(distinct t.session_id)          as sessions,
  avg(iff(f.rating = -1, 1, 0))          as thumbs_down_rate,
  avg(iff(t.abstained, 1, 0))            as abstain_rate,
  avg(iff(t.status = 'error', 1, 0))     as error_rate,
  approx_percentile(t.latency_ms, 0.95) as p95_latency_ms
from obs.chat_turns t
left join obs.feedback f using (turn_id)
group by day;
```

Volume turns, sessions

Quality thumbs down, abstains

Speed p95 latency

Reliability error rate

Cost per answer, from usage views

abstained and status are two columns added to obs.chat_turns in this module.

Let Snowflake watch the assistant for you

ALERT

```
create or replace alert obs.quality_alert
warehouse = ops_wh
schedule = '60 minute'
if (exists (select 1 from obs.daily_health
            where day = current_date()
            and thumbs_down_rate > 0.15))
then call system$$send_email('ops_email_int',
                             'ai-team@neonexus.example',
                             'Assistant quality alert',
                             'Thumbs-down rate above 15% today');
alter alert obs.quality_alert resume;
```

RESOURCE MONITOR

```
create resource monitor app_wh_monitor
with credit_quota = 100 frequency = monthly
start_timestamp = immediately
triggers on 80 percent do notify
on 100 percent do suspend;
alter warehouse app_prod_wh set resource_monitor = app_wh_monitor;
```

Quality and reliability

Alerts on thumbs-down rate, error rate and p95 latency, each against a threshold set from the baseline.

Cost

Resource monitors cap warehouses; budgets cover AI tokens and search serving.

When something goes wrong, start here

Symptom	First check	Usual fix
Answers ignore new programme terms	dbt run status; search service refresh lag	rerun dbt; check change tracking and TARGET_LAG
Assistant is slow	p95 latency; warehouse queueing	size the query warehouse; smaller model for routing
Errors spike	status and error columns in obs.chat_turns	model deprecated or throttled: switch model config
Thumbs-down spike	failure classification (Module 3)	fix the rung the failures point to
Cost spike	Cortex usage views by user and model	cap context, cache, tier models; set budgets

Rehearse it: a runbook nobody has practised is a document, not a capability.

Twelve checks before the first real user

Build

- ☐ Code in Git, reviewed
- ☐ Pinned packages
- ☐ DEV, TEST, PROD from one codebase
- ☐ Pipeline deploys; nobody deploys by hand

Quality and security

- ☐ Evaluation gate passing on every slice
- ☐ Adversarial cases all pass
- ☐ Least-privilege roles granted
- ☐ Log masking and retention in place

Operations

- ☐ Health view on the admin page
- ☐ Alerts routed to a named owner
- ☐ Resource monitor and budgets set
- ☐ Rollback rehearsed

Take the Neo Nexus assistant to production

- | | | |
|----------|---|--------|
| 1 | Repository
restructure into app/, lib/, pages/; add snowflake.yml | 15 min |
| 2 | Pipeline
workflow deploys to TEST with the Snowflake CLI | 20 min |
| 3 | Gate
wire run_gate.py; push a worse prompt; watch it fail | 20 min |
| 4 | Roles
create the four roles; test as CRM_ANALYST | 15 min |
| 5 | Operate
health view, one alert, then a timed rollback drill | 20 min |

Deliverables

- A green pipeline run to TEST, and a red one blocked by the gate
- Four roles, with a screenshot of what CRM_ANALYST can see
- obs.daily_health and one resumed alert
- Rollback drill: what you rolled back, and how long it took

Stretch: switch deployment to the Snowflake Git repository path

Questions worth arguing about

1. A change passes every gate and still makes users unhappy. What did the gate miss, and how do you catch it next time?
2. Should a business user be allowed to edit a prompt in production without a pull request?
3. Who should be paged at 2 a.m. when the assistant starts giving wrong answers?
4. How does this pipeline change when the assistant also answers in Slack and other apps?



References

Snowflake docs: Create and deploy Streamlit apps using Snowflake CLI

Snowflake docs: Snowflake CLI GitHub Action

Snowflake docs: CREATE GIT REPOSITORY; Git integration

Snowflake docs: Alerts; resource monitors; budgets

Google SRE book: Monitoring distributed systems; postmortem culture

SnowPro prep: access control, resource monitors and alerts topics

Cortex beyond Snowflake



REST APIs

call Cortex from any service, language or platform



Agents and MCP

Cortex Agents as tools for other AI systems, and the reverse



Where people work

Slack, Teams, BI tools and custom web front ends

Before Module 6: create a programmatic access token for your user, and have Slack or Teams admin access in a test workspace if you can.